浙江大学
ZHEJIANG UNIVERSITY



边缘计算开发实践
实验报告

实验名称	IMU_GestureDetection
实验地点	玉泉高分子楼-401
姓　名	郑智瑄
学　号	3230103913
实验日期	October 15, 2025
指导老师	皇甫江涛

Contents

1 实验原理 **1**

1.1 **IMU** 1

1.2 机器学习算法 1

2 实验目的 **2**

3 实验任务 **2**

4 实验步骤 **2**

4.1 手势训练数据采集 2

4.2 使用提供的数据集进行机器学习模型训练 3

4.2.1 训练步骤 3

4.2.2 数据可视化 4

4.2.3 训练结果 4

4.2.4 实验结论 4

4.2.5 模型优化 5

4.3 使用自己采集的数据集进行训练 6

4.3.1 数据可视化 6

4.3.2 训练结果 7

4.3.3 实验结论 7

4.4 模型部署与手势识别验证 7

4.4.1 实验步骤 7

4.4.2 实验结果 8

5 实验总结 **8**

1 实验原理

本实验的核心原理是利用惯性测量单元（IMU）捕捉手部动作产生的时序数据，并通过机器学习方法训练一个神经网络模型，最终将该模型部署到微控制器（Arduino Nano 33 BLE Sense）上，以实现对手势的实时分类与识别。

1.1 IMU

IMU 是惯性测量单元（Inertial Measurement Unit）的缩写。它是一种电子设备，用于测量和报告物体的三个基本线性运动（加速度）和三个基本角运动（角速度）。IMU 无论大小从超小型的 MEMS 传感器，到高精度的激光陀螺，其原理都是采用惯性定律实现的，IMU 通常包括一组加速度计和陀螺仪，他们是惯性系统的核心部件，是影响惯性系统性能的主要因素。此外，有时还包括磁力计。

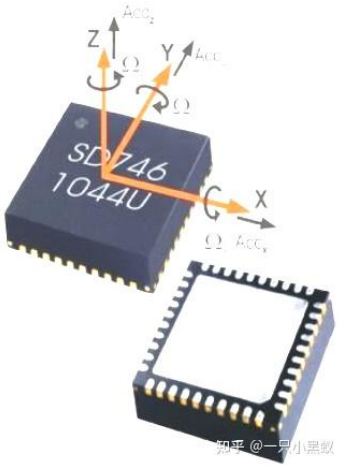


Figure 1: IMU 示意图

1.2 机器学习算法

实验采用 TensorFlow 高级 API Keras 构建一个序贯（Sequential）模型。这是一个典型的多层感知机（MLP），其结构如下：

输入层：隐式定义，接收 714 个归一化后的特征值。

第一个隐藏层：包含 50 个神经元的全连接层（Dense Layer），使用 ReLU（Rectified Linear Unit）作为激活函数。

第二个隐藏层：包含 15 个神经元的全连接层，同样使用 ReLU 激活函数。

输出层：包含 2 个神经元（对应两种手势）的全连接层，使用 Softmax 作为激活函数。Softmax 函数将输出层的原始分数（logits）转换为一个概率分布，每个神经元的输出值表示对应手势的预测置信度。

2 实验目的

利用 Arduino 实现手势识别。

3 实验任务

1. 手势训练数据采集
2. 机器学习模型训练
3. 模型部署与手势识别验证

4 实验步骤

4.1 手势训练数据采集

1. 在 Arduino IDE 中，打开 IMU_Capture.ino。
2. 将程序编译并上传至 Arduino Nano 33 BLE Sense 开发板。
3. 为熟悉数据特征，可先打开串口绘图仪 (Tools -> Serial Plotter) 。按下按钮并执行手势，观察实时输出的传感器数据波形图。完成后关闭串口绘图仪。

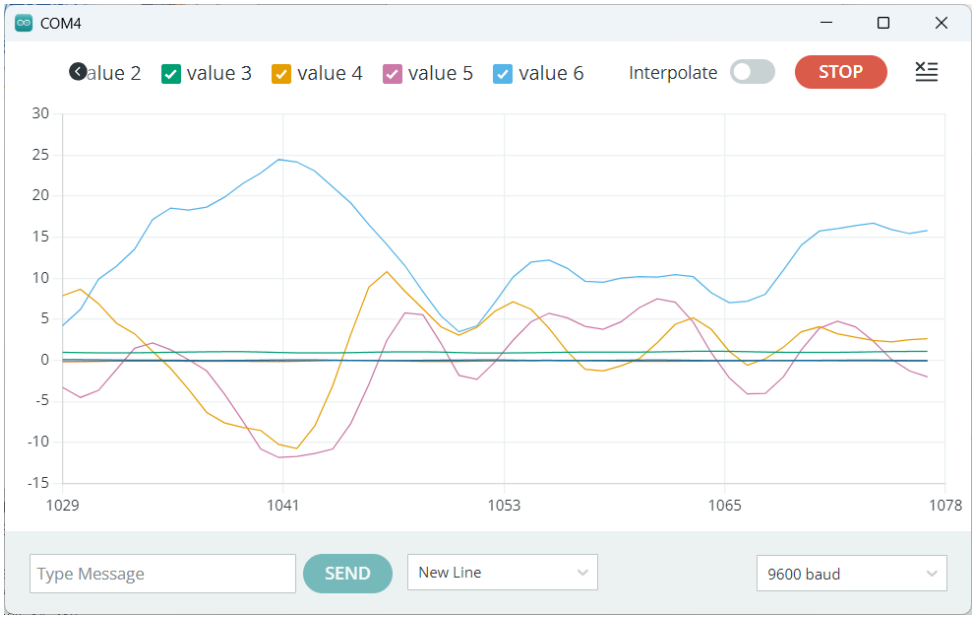


Figure 2: 串口绘图仪查看数据示意图

4. 打开串口监视器 (Tools -> Serial Monitor) 。

5. 采集“挥拳 (punch)”手势数据：按下按钮，然后做出挥拳手势。重复此操作约 10 次以收集足够的数据。

6. 将串口监视器中输出的所有数据从头到尾完整地复制, 并粘贴到一个新的文本文件中，命名为 punch_new.csv。

7. 采集“弯曲 (flex)”手势数据：再次打开串口监视器，按下按钮，然后做出弯曲手势。同样重复约 10 次。

8. 将串口监视器中输出的所有数据复制并粘贴到一个新的文本文件中，命名为 flex_new.csv 。



Figure 3: 串口监视器查看数据示意图

4.2 使用提供的数据集进行机器学习模型训练

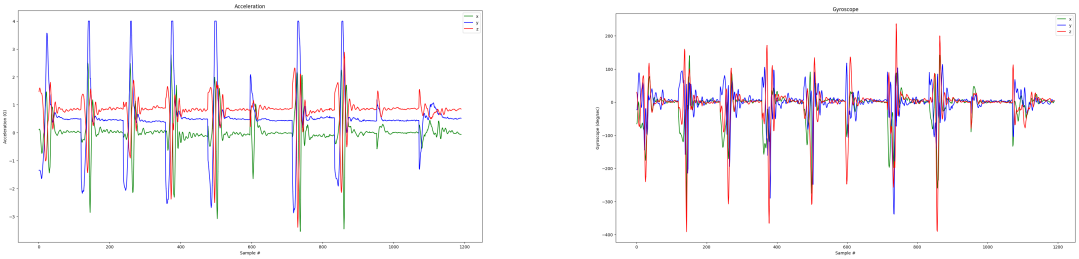
4.2.1 训练步骤

1. 打开 arduino_tinyml_workshop.ipynb 文件

2. 依次执行笔记本中的代码单元格：

   - Setup Python Environment：运行此单元以安装 TensorFlow 等必要的 Python 库。
   - Parse and prepare the data：运行此单元以加载、解析、归一化并分割数据集。代码会将数据分为 60% 的训练集、20% 的测试集和 20% 的验证集。
   - Build & Train the Model：运行此单元以构建神经网络模型并进行 600 个周期的训练。

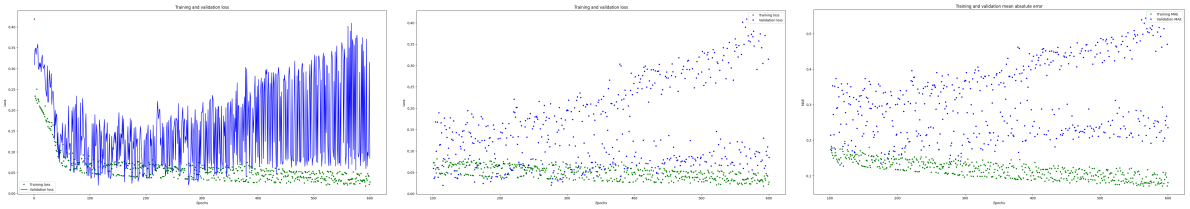
4.2.2 数据可视化



(a) Acceleration (G) (b) Gyroscope (deg/sec)

Figure 4: punch 数据可视化

4.2.3 训练结果



(a) Training and validation loss (b) Skip start (c) Training and validation mae

Figure 5: 训练结果

4.2.4 实验结论

1. **模型收敛趋势良好：**从绿色散点（训练损失）可以看出，随着训练周期的增加，损失值从最初的约 0.23 迅速且持续地下降，最终趋近于 0.0。这表明模型很好地学习了训练数据的特征，成功拟合了训练样本。

2. **存在明显的过拟合 (Overfitting) 现象：**在训练初期（约前 100 个周期），验证损失（蓝色线条）随着训练损失一同快速下降，说明模型在开始阶段学习到了普适的规律。然而，在大约 100-150 个周期之后，训练损失仍在持续降低，但验证损失不再呈现稳定的下降趋势，反而开始出现剧烈且无规律的震荡，并且其值整体上远高于训练损失。这种训练损失很低、但验证损失很高且不稳定的现象是典型的过拟合标志。这意味着模型过度学习了训练数据中的细节和噪声，而没有学到足够通用的模式，导致其在新（验证）数据上的表现不佳。

3. **验证过程表现出高方差 (High Variance)：**蓝色线条（验证损失）的剧烈波动表明模型在不同批次的验证数据上表现非常不稳定

4.2.5 模型优化

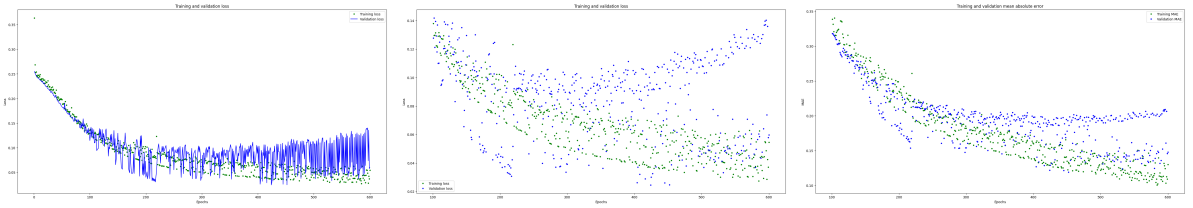
为了改善过拟合问题，选择如下代码简化模型结构，减少隐藏层神经元数量，通过对比可以发现，简化后的模型在训练和验证集上的表现更加接近，验证损失曲线更加平滑，说明模型的泛化能力有所提升，过拟合现象得到缓解。

Listing 1: 原先的 build 代码

```
# build the model and train it
model = tf.keras.Sequential()
model.add(tf.keras.layers.Dense(50, activation='relu'))
model.add(tf.keras.layers.Dense(15, activation='relu'))
model.add(tf.keras.layers.Dense(NUM_GESTURES, activation='softmax'))
model.compile(optimizer='rmsprop', loss='mse', metrics=['mae'])
history = model.fit(inputs_train, outputs_train, epochs=600,
    batch_size=4, validation_data=(inputs_validate, outputs_validate))
```

Listing 2: 简化后的 build 代码

```
# build the model and train it
model = tf.keras.Sequential()
model.add(tf.keras.layers.Dense(10, activation='relu'))
model.add(tf.keras.layers.Dense(5, activation='relu'))
model.add(tf.keras.layers.Dense(NUM_GESTURES, activation='softmax'))
model.compile(optimizer='rmsprop', loss='mse', metrics=['mae'])
history = model.fit(inputs_train, outputs_train, epochs=600,
    batch_size=4, validation_data=(inputs_validate, outputs_validate))
```



(a) Training and validation loss (b) Skip start (c) Training and validation mae

Figure 6: 优化后结果

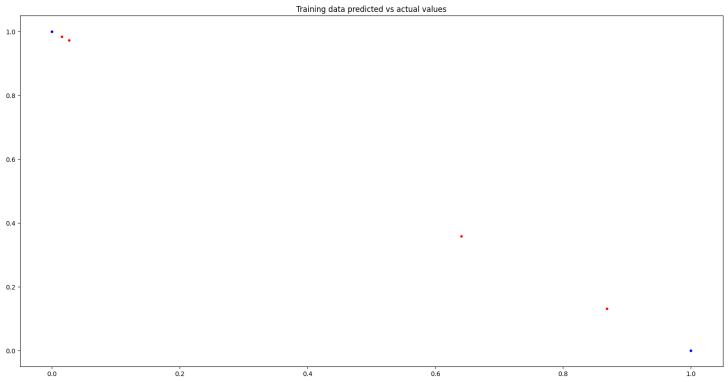


Figure 7: 优化后 Training data predicted vs actual values

4.3 使用自己采集的数据集进行训练

将 Build & Train the Model 处代码更改回原先的代码

更改 Graph Data (optional) 中代码如下

Listing 3: 原先的 build 代码

```
1    filename = "punch_new.csv"
```

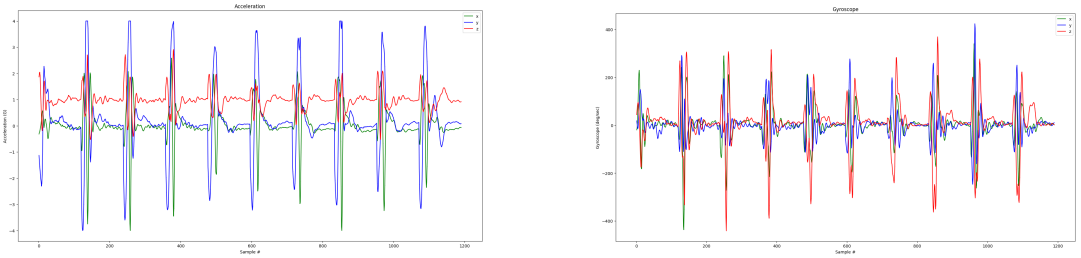
更改 Parse and prepare the data 处代码如下

Listing 4: 原先的 build 代码

```
1  GESTURES = [
2      "punch_new",
3      "flex_new",
4  ]
```

以采用自己采集的数据集进行测试

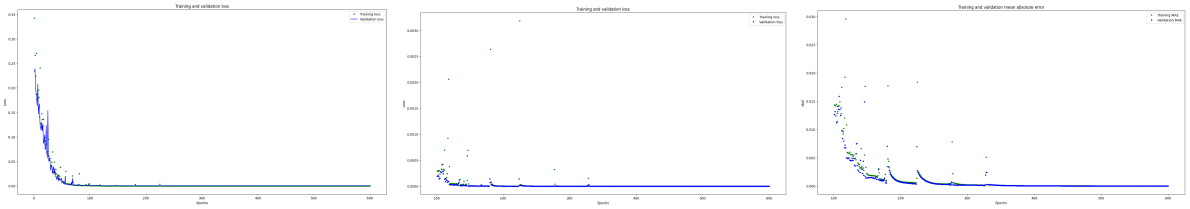
4.3.1 数据可视化



(a) Acceleration (G)　　(b) Gyroscope (deg/sec)

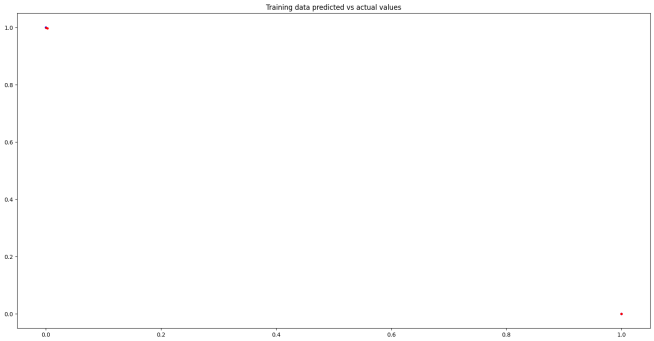
Figure 8: punch 数据可视化

4.3.2 训练结果



(a) Training and validation loss (b) Skip start (c) Training and validation mae

Figure 9: 训练结果




```
predictions =
 [[0.003 0.997]
 [0.    1.   ]
 [1.    0.   ]
 [1.    0.   ]]
actual =
 [[0. 1.]
 [0. 1.]
 [1. 0.]
 [1. 0.]]
```

Figure 10: Training data predicted vs actual values

4.3.3 实验结论

从 Figure 9 中可以看出，训练损失（Training loss）与验证损失（Validation loss）曲线在整个训练过程中高度重合，且均能快速、稳定地收敛至一个非常低的值。这表明模型成功学习到了自采集数据的特征。

从 Figure 10 中可以看出，红点与两端蓝点非常接近，说明在测试集中，预测值与实际值高度一致，模型训练良好。

实验表明，优化训练数据质量有时候远比优化模型结构实用，合适的数据可以让一个简单的模型达到很好的效果。

4.4 模型部署与手势识别验证

4.4.1 实验步骤

1. 将上一步生成的 C 头文件内容替换项目中的模型文件
2. 将最终的识别程序 ‘demo3_2_IMU_Classifier‘ 上传到 Arduino 开发板。
3. 打开串口监视器，执行相应手势，观察模型输出的每个手势的置信度分数（0 到 1 之间），验证手势识别的准确性。

4.4.2 实验结果

如下，分别进行三次 punch 动作与 flex 动作，发现置信度分数非常高，说明均能被正确识别

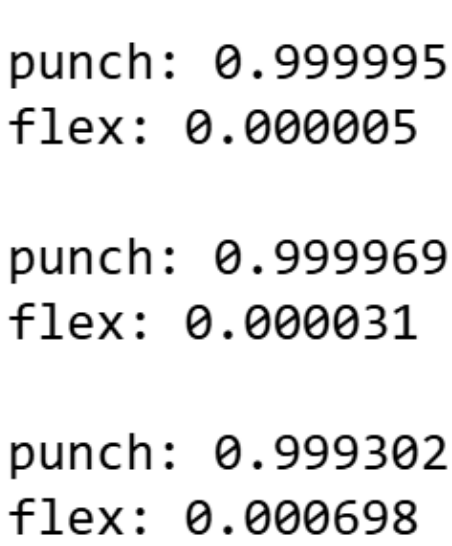


Figure 11: Punch 动作识别结果

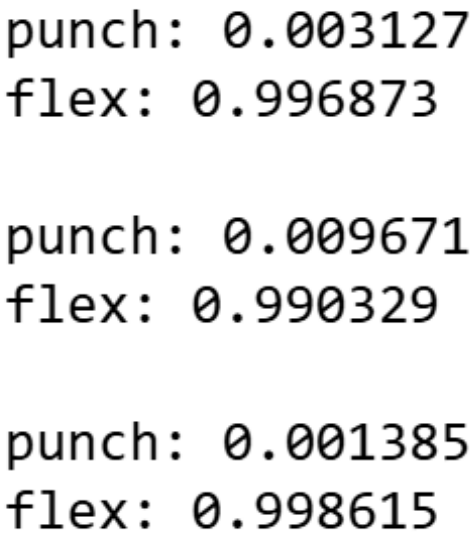


Figure 12: Flex 动作识别结果

Figure 13: 动作识别结果

5 实验总结

该实验成功地在边缘计算设备（Arduino Nano 33 BLE Sense）上，通过 IMU（惯性测量单元）实现了手势的实时识别。

实验表明了训练数据质量和代表性的重要性。一份准确、能够真实反映实际应用场景的数据集，可以让一个相对简单的模型达到优异的性能，